

EventHub

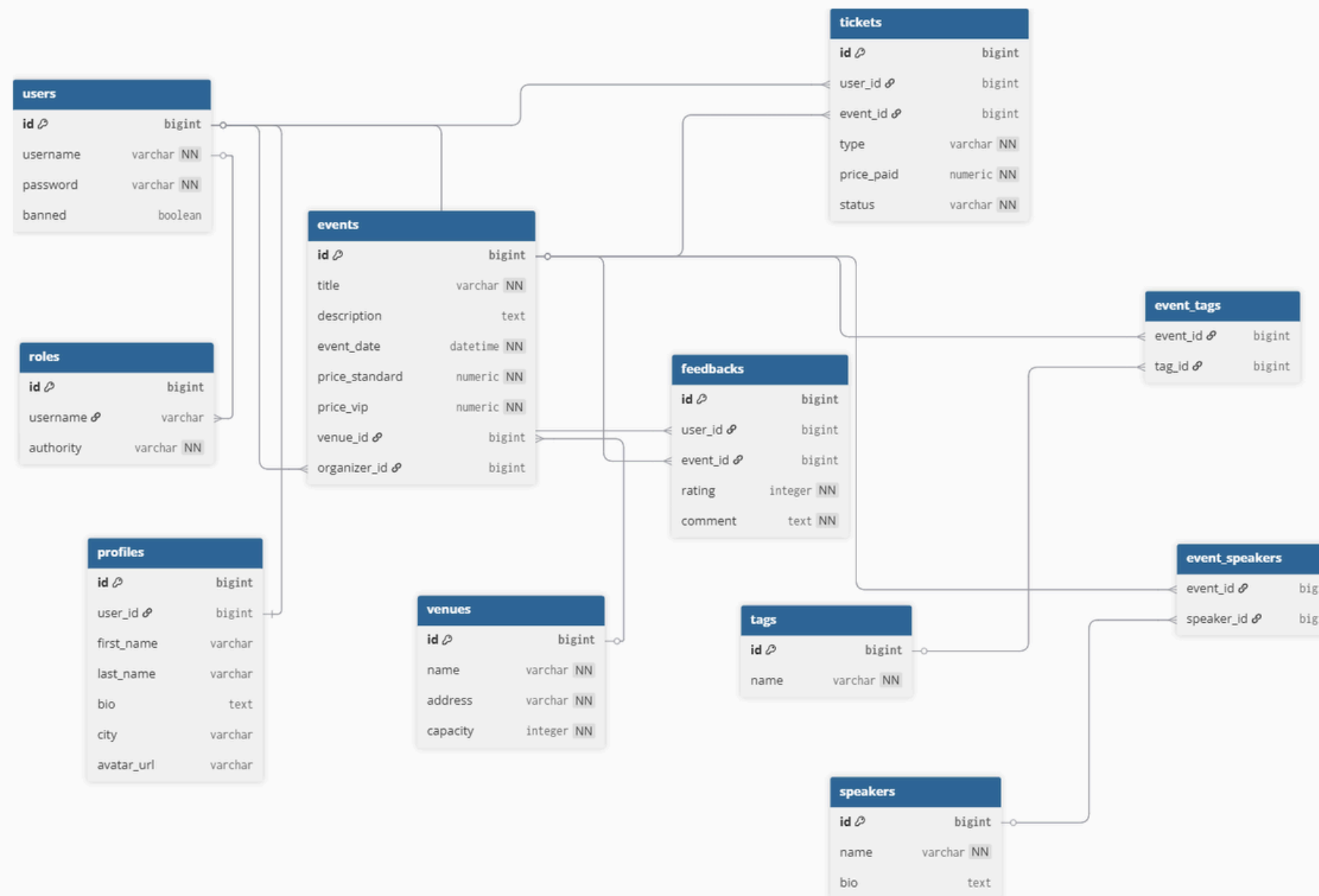
Piattaforma Enterprise per la Gestione e Prenotazione di Eventi

- Architettura: Backend RESTful (Spring Boot) + Client asincrono (Vanilla JS)
- Ruoli Centralizzati: Gestione granulare degli accessi (USER, ORGANIZER, ADMIN)
- Ecosistema Unificato: Digitalizzazione completa del flusso di iscrizione e feedback
- Infrastruttura: Database PostgreSQL isolato e containerizzato in Docker



Partecipante: Francesco Cerrato
Progetto Finale Academy 2026

Il Database e l'ORM



```

@Entity
public class Ticket
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ticket_id")
    private Long id;

    /*
     * In questo caso, la relazione @ManyToOne è unidirezionale.
     * Il codice di relazione è scritto esclusivamente nell'entity Ticket
     * (non nell'entity user)
     */
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "event_id", nullable = false)
    private Event event;

    @Column(nullable = false)
    private BigDecimal pricePaid;

    // Impedisce l'inserimento di testi errati (es. "StAndard" o "Vip-Pass") nel database o nel codice.
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TicketType type; // Utilizza l'enum TicketType

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TicketStatus status; // Utilizza l'enum TicketStatus

    private LocalDateTime createdAt = LocalDateTime.now();

    public Ticket()
    {}
  
```

Il Database e l'ORM

- **Disegno Relazionale:** Database PostgreSQL strutturato con vincoli di integrità referenziale (Foreign Key) e indici ottimizzati.
- **Mappatura ORM:** Spring Data JPA e Hibernate traducono automaticamente le tabelle del database in classi Java e viceversa.
- **Relazioni Complesse:**
 - @OneToOne disaccoppiata tra l'entità User e il suo Profile per la separazione dei dati sensibili.
 - @ManyToOne unidirezionale tra Event e Venue per associare ogni evento alla sua sede specifica.
- **Join Entity Avanzata:** La classe Ticket evolve una classica relazione @ManyToMany tra utenti ed eventi in un'entità autonoma.
- **Integrità dello Stato:** Utilizzo dell'enum TicketStatus mappato a DB come stringa per tracciare in modo rigido gli stati ACTIVE e CANCELLED.

Il Data Access Layer:

3A

Query Derivate e Ricerca Dinamica

Package Repository e Specification

- **Query Derivate Automatiche:** Ereditarietà da JpaRepository per generare codice SQL a runtime basandosi esclusivamente sulla convenzione dei nomi dei metodi.
- **Logica Predittiva:** Verifica immediata dei vincoli di business direttamente nello strato di persistenza (es. prevenzione delle doppie prenotazioni).
- **Filtri Dinamici e Disaccoppiati:** Introduzione del pattern Specification basato sull'API Criteria di JPA per gestire query flessibili senza SQL nativo hardcoded.
- **Criteri Combinabili:** Astrazione programmatica dei filtri di ricerca (data, tag, venue) isolati in metodi statici riutilizzabili.
- **Sicurezza Nativa:** Prevenzione totale dei tentativi di SQL Injection grazie alla tipizzazione forte dei parametri e alla compilazione controllata dei predicati.

Il Data Access Layer:

3B

Query Derivate e Ricerca Dinamica

Package Repository e Specification

```
/*
    Questo repository è iniettato all'interno di EventServiceImpl per countByEventIdAndStatus

    Mentre è iniettato in TicketServiceImpl per prenotare e cancellare un ticket (.save e .delete)
*/
public interface TicketRepository extends JpaRepository<Ticket, Long>
{
    // Questo metodo serve per il punto 7 della traccia: contare i ticket attivi per calcolare i posti disponibili
    /*
        Hibernate, scomponendo l'intero nome del metodo (count By, EventId, And, TicketStatus),
        genera automaticamente una query di questo tipo:
        SELECT COUNT(*) FROM Ticket WHERE EventId = ? AND status = ?
    */
    long countByEventIdAndStatus(Long eventId, TicketStatus status);

    // Verifica se esiste già un ticket ATTIVO associato a uno specifico utente e a uno specifico evento
    boolean existsByUserIdAndEventIdAndStatus(Long userId, Long eventId, TicketStatus status);

    // Recupera i biglietti filtrando per l'username dell'utente
    List<Ticket> findByUserUsername(String username);
}
```

Ogni metodo restituisce una "Specification". Una Specification accetta tre parametri:
root: rappresenta l'entità di partenza (in questo caso 'Event'), serve per selezionare i campi (es. root.get("title")).
query: permette di modificare la struttura della query (es. aggiungere un ORDER BY o un DISTINCT).
criteriaBuilder: usata per creare le condizioni logiche (equal, between, like, ecc.).

```
//
public class EventSpecifications

/*
    Filtra gli eventi che si svolgono in una data specifica
    (dalla mezzanotte al secondo prima della mezzanotte successiva).
*/
public static Specification<Event> hasDate(LocalDate date) {
    return (root, query, criteriaBuilder) -> {
        // Se l'utente non ha selezionato alcuna data, non applica questo filtro (ritorna un pezzo di query vuoto)
        if (date == null) return null;

        // Trasformazione della data singola (es. 2026-10-15) in un intervallo di tempo completo per coprire l'intera giornata
        LocalDateTime startOfDay = date.atStartOfDay(); // 2026-10-15T00:00:00
        LocalDateTime endOfDay = date.atTime(23, 59, 59); // 2026-10-15T23:59:59

        // Genera l'equivalente SQL: WHERE event_date BETWEEN startOfDay AND endOfDay
        return criteriaBuilder.between(root.get("eventDate"), startOfDay, endOfDay);
    };
}

/*
    Filtra gli eventi in base all'ID della sede (Venue).
*/
public static Specification<Event> hasVenueId(Long venueId) {
    return (root, query, criteriaBuilder) -> {
        // Se l'utente non ha selezionato alcuna sede, non applica questo filtro (ritorna un pezzo di query vuoto)
        if (venueId == null) return null;

        // Navighiamo dentro la relazione: partiamo da Event (root), entriamo nel campo "venue" e prendiamo il suo "id"
        // Genera l'equivalente SQL: WHERE venue_id = venueId
        return criteriaBuilder.equal(root.get("venue").get("id"), venueId);
    };
}
```


Data Transfer Object e Validazione applicativa

- **Pattern DTO:** Disaccoppiamento strutturale tra i modelli di persistenza del DB e i dati scambiati tramite le API REST
- **Jakarta Bean Validation:** Uso di annotazioni dichiarative sul DTO per bloccare input non conformi prima dell'elaborazione
- **Vincoli di Presenza:** @NotBlank per impedire l'invio di stringhe vuote o composte da soli spazi su campi obbligatori
- **Controllo di Capacità:** @Size per forzare la lunghezza massima consentita ed evitare eccezioni di troncamento sul DB
- **Messaggi Localizzati:** Feedback d'errore personalizzati restituiti direttamente in formato JSON al client in caso di fallimento

```
public class ProfileUpdateDto
{
    @NotBlank(message = "Il nome non può essere vuoto")
    private String firstName;
    @NotBlank(message = "Il cognome non può essere vuoto")
    private String lastName;
    @Size(max = 255, message = "La biografia non può superare i 255 caratteri")
    private String bio;
    @NotBlank(message = "La città non può essere vuota")
    private String city;
    @Size(max = 500, message = "L'URL dell'avatar è troppo lungo")
    private String avatarUrl;

    public ProfileUpdateDto()
    {}

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}
```

Core Business Logic: Gestione delle Prenotazioni^{5A}

- **Contesto Transazionale:** Metodo marcato con @Transactional per garantire l'atomicità ed eseguire il rollback automatico in caso di anomalie
- **Risoluzione delle Risorse:** Recupero sicuro delle entità Event e User tramite ID e Username con gestione centralizzata delle eccezioni.
- **Vincolo Temporale:** Blocco immediato dei tentativi di prenotazione per eventi svoltisi nel passato o già iniziati.
- **Prevenzione Duplicati:** Verifica preventiva sul database per impedire acquisti multipli di biglietti attivi da parte dello stesso utente.
- **Integrità della Capienza:** Controllo in tempo reale dei posti residui per non superare il limite fisico della struttura ospitante.

```
@Override
@Transactional
public TicketResponseDto createTicket(Long eventId, TicketRequestDto ticketRequestDto, String currentUsername)
{
    // Recupero evento
    Event foundEvent = eventRepository.findById(eventId)
        .orElseThrow( () -> new ResourceNotFoundException("Evento non trovato con id: " + eventId ) );

    // Recupero utente loggato
    User foundUser = userRepository.findByUsername(currentUsername)
        .orElseThrow( () -> new ResourceNotFoundException("Utente non trovato con username: " + currentUsername));

    // Controllo che il Ticket da prenotare non appartenga (relazione) ad un evento svolto nel passato. Step 8 punto 1
    if (foundEvent.getEventDate().isBefore(LocalDateTime.now()))
    {
        throw new IllegalStateException("Impossibile prenotare: l'evento è già iniziato o si è concluso.");
    }

    // 4. Controllo doppia prenotazione. Step 8 punto 2
    if (ticketRepository.existsByUserIdAndEventIdAndStatus(foundUser.getId(), eventId, TicketStatus.ACTIVE))
    {
        throw new IllegalStateException("Hai già una prenotazione attiva per questo evento. Non puoi prenotare due volte.");
    }

    // Verifica posti disponibili
    int seatsLeft = eventService.getAvailableSeats(eventId);
    if (seatsLeft <= 0)
    {
        throw new IllegalStateException("Impossibile prenotare: i posti per questo evento sono esauriti.");
    }
}
```

Core Business Logic: Gestione delle Prenotazioni^{5B}

- **Pricing Dinamico:** Calcolo algoritmico della tariffa finale basato sulla tipologia di biglietto richiesta dall'utente (VIP o STANDARD)
- **Precisione Finanziaria:** Utilizzo del tipo di dato BigDecimal per l'attributo pricePaid, azzerando i rischi di approssimazione numerica
- **Popolamento dello Stato:** Istanziamento dell'entità Ticket configurando lo stato configurando lo stato iniziale su ACTIVE tramite l'enum TicketStatus e iniettando le relazioni Lazy.
- **Persistenza Sicura:** Scrittura del record sul database PostgreSQL tramite il metodo .save() della repository
- **Disaccoppiamento in Uscita:** Conversione finale dell'entità persistita in un TicketResponseDto per isolare lo stato dati dalla risposta di rete

```
// Calcolo automatico del prezzo
BigDecimal finalPrice;
if (ticketRequestDto.getType() == TicketType.VIP)
{
    finalPrice = foundEvent.getVipPrice();
}
else
{
    finalPrice = foundEvent.getPrice();
}

// Creazione e popolamento dell'entità Ticket
Ticket newTicket = new Ticket();
newTicket.setUser(foundUser);
newTicket.setEvent(foundEvent);
newTicket.setStatus(TicketStatus.ACTIVE);
newTicket.setType(ticketRequestDto.getType());
newTicket.setPricePaid(finalPrice);

// Salvataggio del ticket nel DB
Ticket savedTicket = ticketRepository.save(newTicket);

// Conversione in DTO response e return
return convertToResponseDto(savedTicket);
}
```


Transazionalità e Logica di Rimozione (Service Layer)

Gestione delle dipendenze e integrità referenziale

- **Atomicità delle Operazioni:** Uso di @Transactional per garantire il principio ACID: l'intera procedura di eliminazione e annullamento si conclude con successo o fallisce in blocco
- **Integrità delle Authorities:** Cancellazione manuale preventiva dei ruoli associati dall'infrastruttura di Spring Security per evitare eccezioni sui vincoli di chiave esterna (Foreign Key)
- **Annullamento Predittivo:** Scansione preliminare di tutti i ticket attivi acquistati dall'utente per intercettare gli eventi futuri
- **Effetto a Catena:** Variazione automatica dello stato dei biglietti futuri in CANCELLED, preservando lo storico dei dati aziendali
- **Dirty Checking di Hibernate:** Sfruttamento della sessione transazionale attiva per sincronizzare lo stato modificato dei ticket senza invocare metodi .save() ridondanti

```

/*
    Rimuove in modo permanente un utente dal sistema identificandolo tramite l'id.
*/
@Override
@Transactional
public void deleteUser(Long id)
{
    User foundUser = userRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Utente non trovato con id: " + id));

    // Recupero di tutti i ruoli associati allo username di questo utente
    List<Role> rolesToDelete = roleRepository.findByUsername(foundUser.getUsername());

    // Eliminazione record corrispondenti dalla tabella authorities
    for (Role role : rolesToDelete) {
        roleRepository.delete(role);
    }

    // Annullamento automatico di tutti i biglietti attivi per eventi futuri
    // Recuperiamo tutti i ticket attivi dell'utente
    List<Ticket> activeTickets = ticketRepository.findByUserUsername(foundUser.getUsername());

    for (Ticket ticket : activeTickets) {
        // Se lo stato è ACTIVE e l'evento associato deve ancora iniziare (è nel futuro)
        if (ticket.getStatus() == TicketStatus.ACTIVE && ticket.getEvent().getEventDate().isAfter(LocalDate.now())) {
            ticket.setStatus(TicketStatus.CANCELLED);
            // Grazie a @Transactional, le modifiche allo stato del ticket verranno salvate automaticamente
        }
    }

    userRepository.delete(foundUser);
}

```

REST API & Disambiguazione delle Rotte

7

Strato Controller ed estrazione sicura del contesto applicativo

- **Paradigma RESTful:** Endpoint semantici e stateless strutturati tramite l'utilizzo rigoroso dei verbi HTTP appropriati (GET, PUT)
- **Isolamento dell'Identità:** Estrazione sicura dell'utente loggato tramite l'oggetto Principal fornito dal contesto di Spring Security
- **Rotte Autoreferenziali (/me):** Accesso e modifica del profilo blindati lato server, escludendo il passaggio dell'ID client-side
- **Disambiguazione del Routing:** Risoluzione del conflitto strutturale tra costanti statiche ("me") e segnaposto dinamici ("{id}")
- **Privilege Separation:** Isolamento dei percorsi sensibili per ID sotto il prefisso /admin, azzerando i conflitti di parsing a runtime
- **Documentazione Integrata:** Annotazioni @Operation e @ApiResponses di OpenAPI per generare automaticamente contratti d'interfaccia chiari

```
@Operation(  
    summary = "Aggiorna il proprio profilo",  
    description = "Consente all'utente autenticato di modificare i propri dettagli personali " +  
        "come nome, cognome, biografia, città o URL dell'avatar."  
)  
@ApiResponses(value = {  
    @ApiResponse(responseCode = "200", description = "Profilo aggiornato con successo"),  
    @ApiResponse(responseCode = "400", description = "Dati di input non validi o non conformi"),  
    @ApiResponse(responseCode = "401", description = "Utente non autenticato nel sistema"),  
    @ApiResponse(responseCode = "404", description = "Profilo non trovato per l'utente loggato")  
})  
@PutMapping("/me")  
public ResponseEntity<ProfileResponseDto> updateMyProfile (Principal principal,  
    @Valid @RequestBody ProfileUpdateDto inputDto)  
{  
    String loggedUsername = principal.getName();  
  
    ProfileResponseDto updatedProfile = profileService.updateProfileByUsername(loggedUsername, inputDto);  
  
    return ResponseEntity.ok(updatedProfile);  
}
```

```
@Operation(  
    summary = "Recupera un utente tramite ID",  
    description = "Consente di visualizzare le informazioni di un singolo utente cercandolo attraverso il suo ID unico. Riservato all'amministratore."  
)  
@ApiResponses(value = {  
    @ApiResponse(responseCode = "200", description = "Utente trovato e restituito con successo"),  
    @ApiResponse(responseCode = "401", description = "Utente non autenticato"),  
    @ApiResponse(responseCode = "403", description = "Accesso negato: richiesti i permessi di ADMIN"),  
    @ApiResponse(responseCode = "404", description = "Utente non trovato con l'ID fornito")  
})  
@GetMapping("/{id}")  
public ResponseEntity<UserResponseDto> getUserById(@PathVariable Long id)  
{  
    UserResponseDto foundUser = userService.getUserById(id);  
    return ResponseEntity.ok(foundUser);  
}
```

Infrastruttura di Sicurezza

- **Controllo degli Accessi:** Integrazione di Spring Security per proteggere gli endpoint in modo stateless e centralizzato
- **Autenticazione Standard:** Adozione del protocollo HTTP Basic per trasmettere le credenziali codificate all'interno degli header di rete
- **Hashing Forte delle Password:** Utilizzo del componente BCryptPasswordEncoder per l'offuscamento irreversibile dei dati sensibili
- **Integrità del Database:** Protezione crittografica avanzata che impedisce la lettura delle credenziali anche in caso di accesso non autorizzato alla persistenza

```
@Bean
public PasswordEncoder passwordEncoder()
{
    /*
     * Questo bean crea ed attua l'algoritmo BCrypt
     * per criptare le password tramite hash
     */
    return new BCryptPasswordEncoder();
}
```

Gestione Centralizzata delle Eccezioni

9

- **Intercettazione Globale:** Uso dell'annotazione `@RestControllerAdvice` per catturare i fallimenti applicativi lungo tutto il ciclo di vita della richiesta
- **Cattura Chirurgica:** Isolamento delle eccezioni tramite `@ExceptionHandler` per mappare anomalie specifiche (es. `ResourceNotFoundException`)
- **Risposte Standardizzate:** Traduzione delle eccezioni Java in oggetti `ErrorResponse` uniformi e tipizzati, convertiti automaticamente in JSON
- **Integrità dei Flussi:** Forzatura programmatica dello status HTTP ottimale (404 Not Found), evitando l'esposizione di stack trace interni

```
/*
    Punto 7 dello Step 3: Struttura globale per la cattura centralizzata delle eccezioni.
    Questa classe intercetta gli errori lanciati dai Controller e li trasforma in JSON ordinati.
 */
@RestControllerAdvice
public class GlobalExceptionHandler
{
    /*
        Gestione di ResourceNotFoundException.
        Si attiva ogni volta che un utente, un profilo o un elemento non viene trovato.
        Restituisce un codice di stato HTTP 404 Not Found.
    */
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleResourceNotFoundException(
        ResourceNotFoundException exception, WebRequest request)
    {
        // Costruzione DTO di errore con dati eccezione
        ErrorResponse errorResponse = new ErrorResponse(
            LocalDateTime.now(),
            HttpStatus.NOT_FOUND.value(), // 404
            HttpStatus.NOT_FOUND.getReasonPhrase(), // "Not Found"
            exception.getMessage(), // Messaggio personalizzato del service
            request.getDescription(false) // Estrae il path dell'URL (es. uri=/api/users/99)
        );

        // Invio risposta formattata al client
        return new ResponseEntity<>(errorResponse, HttpStatus.NOT_FOUND);
    }
}
```


La Garanzia del Codice: Unit Testing e Mocking Avanzato

Isolamento dei servizi tramite JUnit 5 e Mockito

- **Testing Automatizzato:** Suite di test focalizzata sullo strato dei Servizi per certificare la stabilità dei vincoli applicativi
- **Isolamento dell'Infrastruttura:** Utilizzo di Mockito per simulare il comportamento di Repository e componenti esterni, azzerando le scritture su DB
- **Mocking della Sicurezza:** Configurazione programmatica del SecurityContextHolder per simulare l'identità dell'utente autenticato
- **Prevenzione attacchi IDOR:** Verifica del blocco di sicurezza quando un utente malizioso tenta di manipolare o eliminare risorse non proprie
- **Assertzioni di Eccezione:** Impiego di assertThrows per convalidare il lancio corretto di AccessDeniedException in caso di violazione

```
// NUOVO TEST 7: Verifica il blocco di sicurezza (AccessDeniedException)
@Test
@DisplayName("La cancellazione fallisce con AccessDeniedException se l'utente non è proprietario e non è ADMIN")
void deleteTicket_ThrowsException_WhenNotOwnerAndNotAdmin() {

    Long ticketId = 50L;
    String maliciousUsername = "hacker_user"; // Utente non autorizzato
    String ownerUsername = "mario_rossi";

    User owner = new User();
    owner.setUsername(ownerUsername);

    Ticket existingTicket = new Ticket();
    existingTicket.setId(ticketId);
    existingTicket.setUser(owner);
    existingTicket.setStatus(TicketStatus.ACTIVE);

    // MOCK DI SPRING SECURITY: Simuliamo che l'utente loggato sia un semplice USER
    Authentication authentication = Mockito.mock(Authentication.class);
    SecurityContext securityContext = Mockito.mock(SecurityContext.class);
    Mockito.when(securityContext.getAuthentication()).thenReturn(authentication);
    SecurityContextHolder.setContext(securityContext);

    List<GrantedAuthority> authorities = Collections.singletonList(new SimpleGrantedAuthority("ROLE_USER"));
    Mockito.doReturn(authorities).when(authentication).getAuthorities();

    // MOCK BEHAVIOR
    Mockito.when(ticketRepository.findById(ticketId)).thenReturn(Optional.of(existingTicket));

    // CI ASPETTIAMO UN ERRORE DI ACCESSO NEGATO (403)
    assertThrows(AccessDeniedException.class, () -> {
        ticketService.deleteTicket(ticketId, maliciousUsername);
    });

    // Verifica che lo stato rimanga ACTIVE
    assertEquals(TicketStatus.ACTIVE, existingTicket.getStatus());
}
```

Frontend Statico e Interazione Asincrona

Client Architecture: HTML5, CSS3 e Vanilla JavaScript

- **Architettura Leggera:** Interfaccia utente sviluppata senza framework esterni complessi, massimizzando le performance e azzerando i tempi di build
- **Paradigma Asincrono:** Utilizzo dell'API nativa fetch per consumare gli endpoint REST del backend in background senza ricaricare la pagina HTML
- **Codifica delle Credenziali:** Impiego del metodo nativo btoa() per convertire le credenziali in formato Base64, strutturando l'header standard Authorization con prefisso Basic
- **Session Management:** Memorizzazione dello stato dell'utente (Header, Username e Array dei Ruoli) nel sessionStorage del browser ad autenticazione avvenuta
- **Routing e UI Reattiva:** Gestione centralizzata dei codici di stato HTTP (401 Unauthorized, 403 Forbidden) per mostrare feedback mirati e reindirizzare l'utente su index.html

```
/*
  GESTIONE DEL LOGIN (Basic Auth + Profilazione Ruoli)
*/
async function handleLogin(event) {
  event.preventDefault(); // Blocca il ricaricamento della pagina HTML

  const username = document.getElementById('loginUsername').value;
  const password = document.getElementById('loginPassword').value;

  // Codifica le credenziali in formato Base64 per lo standard HTTP Basic Auth
  const credentials = btoa(`${username}:${password}`);
  const authHeaderValue = `Basic ${credentials}`;

  try {
    // Interrogiamo una rotta protetta del backend per testare se le credenziali sono valide.
    // Recuperiamo il profilo dell'utente dall'endpoint /me per estrarre i ruoli reali (Step 3.4)
    const response = await fetch(`${API_BASE_URL}/api/users/me`, {
      method: 'GET',
      headers: {
        'Authorization': authHeaderValue,
        'Accept': 'application/json'
      }
    });

    if (response.ok) {
      const userData = await response.json();

      // Se le credenziali sono giuste, salviamo l'header, lo username e l'array dei ruoli nella sessione del browser
      sessionStorage.setItem('authHeader', authHeaderValue);
      sessionStorage.setItem('username', username);
      sessionStorage.setItem('roles', JSON.stringify(userData.roles || []));

      alert('Login effettuato con successo!');
      window.location.href = 'index.html'; // Sposta l'utente sulla Home come utente loggato
    } else if (response.status === 401) {
      alert('Credenziali non corrette. Riprova.');
```

Profilazione e Navbar Dinamica (Client Layer)

11B

Gestione dell'esperienza utente in base ai ruoli autorizzativi

- **User Experience Adattiva:** Modifica dinamica dell'interfaccia grafica in tempo reale in base ai privilegi specifici dell'account autenticato
- **Lettura dello Stato:** Ispezione programmatica dell'array dei ruoli precedentemente salvato all'interno del sessionStorage del browser
- **Separazione Visiva dei Ruoli:**
 - Visualizzazione del pannello di controllo globale riservata esclusivamente agli amministratori (ROLE_ADMIN).
 - Accesso ai moduli di creazione del catalogo abilitato unicamente per gli organizzatori (ROLE_ORGANIZER).
- **Manipolazione del DOM:** Utilizzo delle proprietà JavaScript .style.display o .classList per mostrare o nascondere le voci sensibili della navbar
- **Sicurezza di Superficie:** Protezione visiva lato client che si affianca al controllo e alla validazione invalicabile operata lato server

```
//  
function updateNavbar() {  
  const isLoggedIn = sessionStorage.getItem('authHeader') !== null;  
  let roles = [];  
  
  // Tenta di estrarre l'elenco dei ruoli salvati in formato stringa JSON  
  try {  
    roles = JSON.parse(sessionStorage.getItem('roles')) || [];  
  } catch(e) {  
    roles = []; // Fallback in caso di sessione vuota  
  }  
  
  // Recuperiamo tutti gli elementi della navbar tramite il loro ID  
  const navLogin = document.getElementById('navLogin');  
  const navSignup = document.getElementById('navSignup');  
  const navBookings = document.getElementById('navBookings');  
  const navProfile = document.getElementById('navProfile');  
  const navOrganizer = document.getElementById('navOrganizer');  
  const navAdmin = document.getElementById('navAdmin');  
  const navLogout = document.getElementById('navLogout');  
  
  if (isLoggedIn) {  
    // Se l'utente è loggato, nascondiamo Login e Registrati  
    if (navLogin) navLogin.classList.add('hidden');  
    if (navSignup) navSignup.classList.add('hidden');  
  
    // Mostriamo i link dell'utente autenticato  
    if (navBookings) navBookings.classList.remove('hidden');  
    if (navProfile) navProfile.classList.remove('hidden');  
    if (navLogout) navLogout.classList.remove('hidden');  
  
    // PROFILAZIONE REALE DEI LINK GESTIONALI IN BASE AI RUOLI DEL DB (Step 11/12)  
    if (navOrganizer) {  
      if (roles.includes('ROLE_ORGANIZER') || roles.includes('ROLE_ADMIN')) {  
        navOrganizer.classList.remove('hidden');  
      } else {  
        navOrganizer.classList.add('hidden');  
      }  
    }  
  
    if (navAdmin) {  
      if (roles.includes('ROLE_ADMIN')) {  
        navAdmin.classList.remove('hidden');  
      } else {  
        navAdmin.classList.add('hidden');  
      }  
    }  
  } else {  
    // Se l'utente NON è loggato, mostriamo solo Login, Registrati ed Eventi  
    if (navLogin) navLogin.classList.remove('hidden');  
    if (navSignup) navSignup.classList.remove('hidden');  
  
    // Nascondiamo tutto il resto  
    if (navBookings) navBookings.classList.add('hidden');  
    if (navProfile) navProfile.classList.add('hidden');  
    if (navOrganizer) navOrganizer.classList.add('hidden');  
    if (navAdmin) navAdmin.classList.add('hidden');  
    if (navLogout) navLogout.classList.add('hidden');  
  }  
}
```


Containerizzazione e Build Multistadio

12A

Ottimizzazione dell'immagine di produzione tramite Dockerfile

- **Isolamento dell'Ambiente:** Utilizzo di Docker per pacchettizzare il backend, garantendo l'esecuzione identica del software su qualsiasi macchina host
- **Separazione delle Responsabilità:** Strategia di build articolata su due stadi indipendenti per dividere la fase di compilazione da quella di runtime
- **Stage 1 (Build):** Ambiente completo basato su Maven per scaricare le dipendenze del pom.xml e compilare il file .jar
- **Stage 2 (Run):** Immagine minimale basata sull'ambiente leggero eclipse-temurin:17-jre-alpine
- **Efficienza e Sicurezza:** Riduzione drastica del peso dell'immagine finale ed eliminazione dei compilatori per minimizzare la superficie di attacco

```
# Stage 1: Compilazione del codice sorgente tramite Maven ed JDK 17
FROM maven:3.8.5-openjdk-17 AS build
WORKDIR /app
# Copia i file di configurazione e il codice sorgente nel container
COPY pom.xml .
COPY src ./src
# Compila il progetto saltando l'esecuzione dei test per velocizzare l'avvio del container
RUN mvn clean package -DskipTests

# Stage 2: Creazione dell'immagine finale leggera per l'esecuzione usando Eclipse Temurin
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
# Copia il file .jar generato dallo stage precedente
COPY --from=build /app/target/*.jar app.jar
# Espone la porta interna utilizzata dall'applicazione
EXPOSE 8081
# Comando per avviare l'applicazione Spring Boot
ENTRYPOINT ["java", "-jar", "app.jar"]
```


Orchestrazione dei Servizi

12B

Coordinamento e controllo dello stato di salute dei container

- **Orchestrazione Centralizzata:** Uso di Docker Compose per coordinare e configurare in un unico file dichiarativo i servizi interconnessi
- **Persistenza Garantita:** Configurazione di un Docker Volume esterno ("postgres_data") per non perdere i dati delle tabelle allo spegnimento dei container
- **Healthcheck Programmatico:** Monitoraggio continuo tramite l'utility pg_isready per verificare la reale reattività del DBMS PostgreSQL
- **Risoluzione della Race Condition:** Blocco dell'avvio anticipato dell'applicazione tramite la clausola depends_on con condizione di salute
- **Iniezione delle Proprietà:** Disaccoppiamento delle credenziali del database forzate a runtime tramite variabili d'ambiente sovrascritte

```
cris@hane:~/OneDrive/Desktop/EventHub-Academy/12B/docker-compose.yml$ cat docker-compose.yml
# Specifica la versione della sintassi di Docker Compose utilizzata.
version: '3.8'
# Il blocco 'services' apre l'elenco dei container che vogliamo accendere.
services:
  # Database PostgreSQL per la memorizzazione dei dati di EventHub
  postgres-db:
    # Indica a Docker quale pacchetto ufficiale scaricare da internet
    image: postgres:15-alpine
    container_name: eventhub-postgres
    ports:
      - "5432:5432" # Espone la porta del DB sul PC locale
    environment:
      POSTGRES_DB: eventhub # Nome del database iniziale
      POSTGRES_USER: eventhub_admin # Username di amministrazione del DB
      POSTGRES_PASSWORD: password123 # Password di accesso
    volumes:
      - postgres_data:/var/lib/postgresql/data # Persistenza dei dati sul disco locale
    # Verifica lo stato di salute del DB prima di far partire altre app collegate
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U eventhub_admin -d eventhub"]
      interval: 5s
      timeout: 5s
      retries: 5
  # Interfaccia web grafica per l'amministrazione visiva del database
  adminer:
    # Scarica l'ultima versione disponibile ('latest') del software Adminer da Docker Hub.
    image: adminer:latest
    container_name: eventhub-adminer
    ports:
      - "8080:8080" # Raggiungibile via browser su http://localhost:8080
    depends_on:
      - postgres-db # Attende l'avvio del database prima di partire
  # Applicazione Spring Boot (EventHub Backend)
  spring-app:
    # Indica a Docker di cercare il file 'Dockerfile' nella cartella corrente per compilare l'app
    build: .
    container_name: eventhub-backend
    ports:
      - "8081:8081" # Raggiungibile via browser o Postman sulla porta 8081 locale
    environment:
      # Configura le variabili d'ambiente lette dal file application.properties
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-db:5432/eventhub
      SPRING_DATASOURCE_USERNAME: eventhub_admin
      SPRING_DATASOURCE_PASSWORD: password123
      SPRING_JPA_HIBERNATE_DDL_AUTO: update
    depends_on:
      postgres-db:
        # Avvia Spring Boot solo quando il database è totalmente pronto e superato l'healthcheck
        condition: service_healthy
volumes:
  postgres_data: # Dichiarazione del volume per non perdere i dati allo spegnimento
```